Functional Programming

:l foo “Haskell”

Markos Vassilis, Mediterranean College

Week 06

Functions, Functions, Functions...

Greater Than 10

- It is well known that all positive integers greater than 10 are boring.
- Write a function that filters a list of integers and keeps only non boring functions:
  - noBore :: [Int] -> [Int]

```
noBore :: [Int] -> [Int]
noBore ls = filter lt10 ls
  where lt10 n = n <= 10 && n > 0
```

- But, why waste our time and define lt10 ? Is there any chance we will use it elsewhere?

Anonymous Functions

- In many cases, you just need a function for a particular occasion but you do not want to pollute the namespace by defining it globally.
  - This what **where** actually does.
- However, why even give it a name?
- This is what anonymous functions are for!

```
noBore :: [Int] -> [Int]
noBore ls = filter (\n -> n <= 10 && n > 0) ls
```

- Syntax: \ <input> **->** <body>
  - The -> (arrow) substitutes “=” in a typical function definition.
  - The \ is intended to look like a lambda (λ) just without the left short left (true story…).

Operator Sections

- Things might be even shorter in some cases.
- Consider this function:

```
aBitBore :: [Int] -> [Int]
aBitBore ls = filter (\n -> n <= 10) ls
```

- Haskell offers a special syntax for the case of such simple functions where we just want to apply a single operator on some input:

```
aBitBore :: [Int] -> [Int]
aBitBore ls = filter (<=10) ls
```

- Essentially, we abstract out the name of the input, since it does not really matter.
- In general, this (?x) thing, where ? is any Haskell operator is called an **Operator Section**. So, it works with any other operator such as <, >, ==, etc.

More Abstraction

- Reconsider this example:

```
aBitBore :: [Int] -> [Int]
aBitBore ls = filter (<=10) ls
```

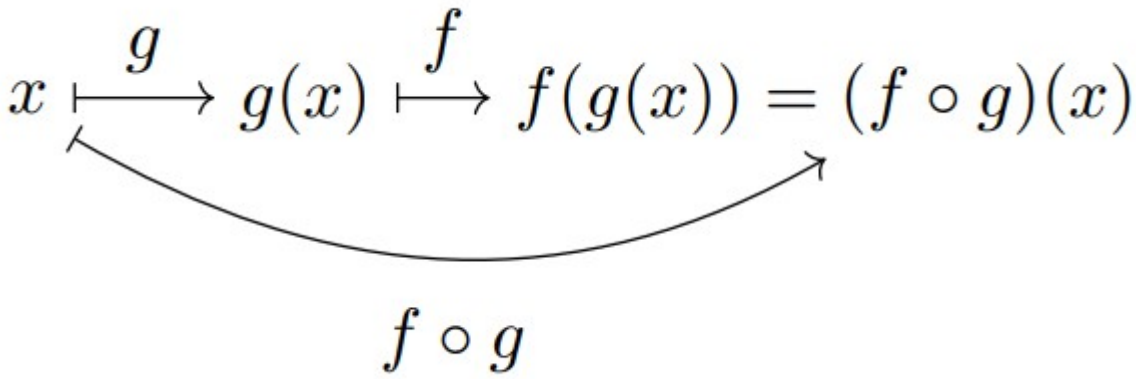
- Why do we need ls?
- Actually, we do not, since we know (type declaration) that this function is applied onto a list of integers.
- So, why even name it?

```
aBitBore :: [Int] -> [Int]
aBitBore = filter (<=10)
```

- This makes our code even more succinct (and somewhat cryptic for non–Haskell people…)

Function Composition

- We have informally talked about function composition in the past.
- A brief maths interlude:



Function Composition

- Since (almost) everything is a function in Haskell, we can think of them as “objects” on which we can operate.
- For instance, assume we want to check whether the length of a list is even or not. How would you implement this in Haskell?

```
evenLn :: [a] -> Bool
evenLn ls = even (length ls)
```

- The above is a typical example of function composition:
  - We first compute the output of length on ls, and then;
  - We then compute the output of even on the previous output.
- Such function “chaining” cases are typical for function composition:

```
evenLn :: [a] -> Bool
evenLn = even . length
```

Your Turn

- Define a function **comp** that implements function composition in the same way that . does.
- The function signature should be as follows:

```
comp :: (b -> c) -> (a -> b) -> (a -> c)
```

- Take some time to think of it...

Currying

[Cringe Alert!] Curry



Steph Curry



Haskell Curry



No Curry

Currying

- Consider the following function:

```
foo x y = x + y
```

- What would be its signature? (there are multiple correct answers).
- Assuming that everything is an integers:

```
foo :: Int -> Int -> Int
```

- Why is that? (Recall freely from previous lectures).
- Essentially, this is an instance of what we call “Currying”.
  - On a side–note, currying was not first invented by Curry (neither Steph, no Haskell), but by Moses Schoenfinkel (the third guy on right in the previous slide).

Currying

- But, what is currying, actually?
- As we saw before, foo, while seeming accepting two arguments, actually accepts a single argument which is an integer and outputs a function taking as input an integer and returning another integer.
- So, Int -> Int -> Int should be read as: Int -> (Int -> Int), but we typically omit the right-most parentheses in such cases.
- This is the quintessence of currying: We can represent any function taking n arguments as a composition of many single–variable functions that return other functions!
- This is really useful at a more formal level, since it allows proving stuff about (efficient) computability of programs

curry And uncurry

- Haskell offers two nice functions regarding currying stuff and uncurrying stuff:
  - curry f x y = f (x, y) → Typically, this makes a function f accept its two arguments as a single tuple, like a “two arguments” function.
  - uncurry f (x, y) = f x y → The opposite of the previous one.
- For instance:
  - uncurry (+) (4, 7) → 5
- Why don’t we have curry uncurry versions for more than two arguments?
- Stay tuned…

Partial Application

- How can currying help us - besides making function declarations seem bogus?
- This is roughly what happens here:

```
bar :: [Integer] -> [Integer]
bar = filter (>3)
```

- We have defined this one through composition, omitting any arguments.
- Since we omit one argument from filter, what does filter return?
  - filter (>3) actually returns a **function** that waits for a list and outputs another list.
  - Exactly what bar is!

Folds

What Do You Observe?

- What do the following functions share in common?

```haskell
sum' :: [Integer] -> Integer
sum' []     = 0
sum' (x:xs) = x + sum' xs

product' :: [Integer] -> Integer
product' [] = 1
product' (x:xs) = x * product' xs

length' :: [a] -> Int
length' []     = 0
length' (_:xs) = 1 + length' xs
```

Even More Abstraction

- In all cases above, each function:
  - Has a base case of how to handle an empty list.
  - Then applies the same operation to each element of the list.
- Consider this list: x == [2, 5, 1, 3]:
  - Remove syntactic sugar: x == 2:(5:(1:(3:[])))
  - Then, in order to compute the sum of its elements:
    - Substitute [] with 0
    - Substitute : with + and leave its elements as they are.
    - Voilà!

Even More Abstraction

- Consider again x == [2, 5, 1, 3]:
  - Remove syntactic sugar: x == 2:(5:(1:(3:[])))
  - Then, in order to compute the product of its elements:
    - Substitute [] with 1
    - Substitute : with * and leave its elements as they are.
    - Voilà!
- Consider again x == [2, 5, 1, 3]:
  - Remove syntactic sugar: x == 2:(5:(1:(3:[])))
  - Then, in order to compute the product of its elements:
    - Substitute [] with 0
    - Substitute : with + and each element with 1.
    - Voilà!

Even More Abstraction

- In all cases above we what we actually do is:
  - substitute [] by some **base value**;
  - substitute the colon with the **operation** we want to perform on each element.
- So, assuming the base value is a piece of data and the operation is a function (of “two arguments”), the pattern above looks something like this:
  - Get a list, e.g., x == [5, 2, 7, 4].
  - Remove syntactic sugar: x == 5:(2:(7:(4:[])))
  - [] → base, : → f (in prefix notation).
  - So: x == f 5 (f 2 (f 7 (f 4 base))).
- This is called **folding**.

Your Turn!

- Implement a (higher order) function that actually implements folding.
- The function's signature should be:

```
fold :: b -> (a -> b -> b) -> [a] -> b
```

- That is, fold gets as parameters:
  - something of type b;
  - a function which gets something of type a and something of type b, returning an output of type b, and;
  - a list of elements of type a;
- and returns something of type b (the same as the base case).

Our `fold`

```
fold :: b -> (a -> b -> b) -> [a] -> b
fold base f []  = base
fold z f (x:xs) = f x (fold z f xs)
```

- Using the above, implement sum, length, and product.

One way of doing so:

```
sum''     = fold 0 (+)
product'' = fold 1 (*)
length''  = fold 0 (\_ s -> 1 + s)
```

foldr, foldl

- Folding is implemented in Prelude in two ways:
  - foldr → right folding, as in our case, just with parameters passed in a different order (function, base value, list)
  - foldl → left folding.
  - Prefer foldl’ from Data.List since it is more efficient!

Fun Time!

Wholemeal Programming

Have some fun with the following lab from UPENN:

https://www.cis.upenn.edu/~cis1940/spring13/hw/04-higher-order.pdf

In case you get stuck, seek for help at: solutions/hof.hs.

Any Questions

Don’t forget to fill in the questionnaire shown right!

Office hours:
Tuesday: 09:00 —10:00, Glyfada, PC Lab
Wednesday: 15:00 — 17:00, Pellinis, 203



https://forms.gle/YU1mjxjBjBf8hyvH7